

Stack - The Quick and Dirty

What's a Stack?

- It's a collection of elements you can only access with two moves: **push** (add) and **pop** (remove).
- The key rule is **LIFO (Last In, First Out)**. Think of a stack of plates—the last one you put on is the first one you grab.
- Real-life examples: stacks of books, plates in a canteen, and Undo/Redo features.

Why Bother with Stacks?

- They are simple and straightforward.
- They offer fast and consistent performance.
- They're used for memory management and handling function calls (like in recursion).

Stack's Personality Traits

- It's a LIFO structure.
- Main operations are Push & Pop.
- All action happens at the **Top of Stack**.
- You can't access random elements; you can only deal with the top one.
- Can be built using Arrays or Linked Lists.

The Main Moves (Operations)

1. **PUSH**: Adds an element to the **top**.
 - a. *Heads up*: Check for **Overflow** (is the stack full?).
2. **POP**: Removes the top element.
 - a. *Heads up*: Check for **Underflow** (is the stack empty?).
3. **PEEK**: Lets you look at the top element *without* removing it.

Stack Pitfalls (Overflow & Underflow)

- **Overflow (✗)**: Happens when you try to PUSH into a full stack.
- **Underflow (✗)**: Happens when you try to POP from an empty stack.

Common Implementations

- **Array-Based Stack:**
 - Uses a fixed size.
 - Memory is contiguous.
 - Simple to implement, but may waste memory.
- **Linked List-Based Stack:**
 - Dynamic size (grows/shrinks).
 - Memory is scattered/dynamic.
 - More flexible and memory efficient, but requires dealing with pointers.

Implementation Showdown: Array vs. Linked List

Feature	Array Stack	Linked List Stack
Size	Fixed	Dynamic
Implementation	Simple	More complex (pointers)
Memory Waste	May waste memory	Efficient usage
Push/Pop Time	O(1) (Super Fast)	O(1) (Super Fast)
Access Time	O(1) (Direct access possible)	O(n) (Requires traversal)

Cool Stack Tricks (Applications)

1. **Reversing Data:** Push everything in, then pop it out (e.g., reversing a string).
2. **Decimal to Binary:** Push remainders (after dividing by 2) to the stack, then pop them out for the binary number.
3. **Delimiter Checking:** Checks if parentheses/brackets are balanced (e.g., $(a + b) - c$).
 - a. Push opening symbols ($($, $\{$, $[$).
 - b. Pop and match closing symbols $)$, $\}$, $]$).
4. **Infix to Postfix Conversion:** Helps manage operator precedence to convert expressions (e.g., converting $A * B$ to AB^*).

Stacks and Function Calls (The Real Deal)

- When a function is called, the system **pushes** a "stack frame" (or activation record) onto the stack.

- This frame saves important stuff like where to return to and local variables.
- When the function is done, the frame is **popped**, and the program returns to the correct spot (**LIFO**).

Watch Out For! (Common Pitfalls)

- **Forgetting Underflow:** Always check if the stack is empty before popping.
- **Stack Overflow:** Don't try to push past the fixed size of an array-based stack.
- **Bad Top Handling:** Make sure you correctly adjust the **top** index/pointer.
- **Unbalanced Push/Pop:** This can lead to a crash.

```
printf("Stack Underflow\n");
```

back to main